FAILURE REPORT

Task Ref. No: Chakra/IPD/Zozo/Actuation/Simulation

Task Description (2-3 lines): Failure analysis of **failedsim.py**, the MuJoCo script meant to load the robot and drive its motors, reviewed against the working reference **sucessfulsim.py**. The script fails its stated purpose: two critical defects (a missing free joint and force-mode actuators) plus seven stability and robustness gaps.

SL no	Failure	Severity
1.	Free joint added after attach() and looked up by name	Critical
2.	Actuators are force motors, not velocity servos	Critical
3.	Unconditional data.qpos[2] shift	High
4.	No force or control limits on the actuators	Medium
5.	No joint damping or armature	Medium
6.	No contact-solver tuning for the floor	Medium
7.	min_z search handles mesh geoms only, no clearance	Medium
8.	Default (Euler) integrator left in place	Medium
9.	Duplicate collision geometry	High

Please see the following pages for a detailed breakdown of each failure.

FAILURE BREAKDOWN

Status: FAILS to meet its stated purpose ("load the robot and drive the motors")
Severity: High
Reference implementation: sucessfulsim.py
Scope: Static review of failedsim.py vs. sucessfulsim.py, cross-checked against MuJoCo MjSpec semantics

Summary

failedsim.py compiles and launches a viewer, and under benign circumstances the robot may be seen standing on the floor, which is why it looks superficially correct. However, the script has **two load-bearing defects** that make it fail whenever a user actually tries to use it, plus a stack of stability and robustness gaps. It is not a working replacement for sucessfulsim.py.

The two show-stoppers are:

1. **The free joint is very likely never attached to the robot**, leaving it welded into the world (or placed incorrectly), because the code searches for the body name "root" after scene_spec.attach(...).
2. **The "velocity motors" are actually constant-gain force motors**, because biastype is never set, so biasprm[2] = -100 is silently ignored. Sliders command force, not velocity, at up to ~500 N·m.

Failure #1: Free joint added after attach() and looked up by name

Severity: Critical **Location:** failedsim.py, steps 5–6

```
scene_spec.attach(urdf_spec, frame=mount_frame)     # <- attach first

for body in scene_spec.bodies:                      # <- then look for "root"
    if body.name == "root":
        body.add_freejoint()
        break
```

What happens: MjSpec.attach() merges the child spec into the parent and, depending on the MuJoCo version and whether the child has a model name (the URDF <robot name="..."> typically supplies one), prefixes child element names (e.g. assembly_1/root). The post-attach loop matches against a name that may no longer exist. If it does not match, no free joint is added and:

- The robot is rigidly fixed to the world at whatever pose the URDF places it (often mid-air).
- The later data.qpos[2] += -min_z shifts an unrelated qpos component, so "drop to floor" does nothing useful.

Contrast: sucessfulsim.py adds the free joint to root_body in the URDF spec, before scene_spec.attach(...), so it lands on the robot body regardless of name prefixing.

Fix: Move the free-joint insertion to before scene_spec.attach(...), operating on urdf_spec.bodies. Then verify after compile:

```
free_joints = [j for j in range(model.njnt)
               if model.jnt_type[j] == mujoco.mjtJoint.mjJNT_FREE]
if not free_joints:
   raise RuntimeError("Free joint was not added — check attach ordering.")
```

Failure #2: Actuators are force motors, not velocity servos

Severity: Critical **Location:** failedsim.py, step 4

```
act.gainprm[0] = 100.0
act.biasprm[1] = 0.0
act.biasprm[2] = -100.0   # <- silently ignored
act.ctrlrange[:] = [-5.0, 5.0]
```

act.biastype is never set. The default is mjBIAS_NONE, which makes biasprm irrelevant. The actuator degenerates to:

```
force = gainprm[0] * ctrl = 100 * ctrl
```

At slider ctrl = 5 the motor commands 500 N·m. Either ctrllimited auto-resolves to true (MuJoCo does this when ctrlrange is set on the spec), in which case the joint gets ±500 N·m and slams into its stop; or it stays false, in which case ctrl is unbounded. Either way, “set motor velocity” does not produce velocity.

Contrast: sucessfulsim.py sets the actuator up as a real velocity servo:

```
actuator.gaintype = mujoco.mjtGain.mjGAIN_FIXED
actuator.gainprm[0] = MOTOR_KV

actuator.biastype = mujoco.mjtBias.mjBIAS_AFFINE
actuator.biasprm[0] = 0.0
actuator.biasprm[1] = 0.0
actuator.biasprm[2] = -MOTOR_KV
```

With AFFINE bias and biasprm = [0, 0, -KV], the actuator solves gainprm[0]·ctrl + biasprm[2]·qvel = 0, i.e. qvel → ctrl.

Fix: Copy those five lines verbatim (adjust MOTOR_KV). Never leave biastype at its default when biasprm is set.

Failure #3: Unconditional data.qpos[2] shift

Severity: High **Location:** failedsim.py, step 9

```
data.qpos[2] += -min_z
```

This assumes the free joint is the first joint in the model, i.e. qpos[2] is the base z. That is true only when the free joint lands on root in traversal order, which is exactly what Failure #1 often prevents. The moment #1 is fixed for a different URDF ordering, or a hinge appears before the free joint, this line corrupts a hinge angle instead of the base z.

Contrast: sucessfulsim.py locates the free joint by type after compile and uses the model's joint-to-qpos address:

```
free_id = next(
   j for j in range(model.njnt)
   if model.jnt_type[j] == mujoco.mjtJoint.mjJNT_FREE
   and "root" in (mujoco.mj_id2name(
      model, mjOBJ_BODY, model.jnt_bodyid[j]) or "").lower())
base_qpos = model.jnt_qposadr[free_id]
data.qpos[base_qpos + 2] += -min_z + FLOOR_CLEARANCE
```

It also includes a fallback if the substring check fails, plus a hard error if no free joint is found.

Fix: Use model.jnt_qposadr[free_id]. Never index qpos by a literal offset.

Failure #4: No force or control limits on the actuators

Severity: Medium **Location:** failedsim.py, step 4

act.forcelimited is never set, so forcerange is not enforced. Combined with Failure #2, the actuator can output gain * ctrl with no upper bound, up to ±500 N·m from a slider drag. Even after #2 is fixed, without a torque cap the servo tries to reach the commanded velocity regardless of torque, and with missing damping/armature (#5) the solver produces spikes.

Fix: Match sucessfulsim.py:

```
act.forcelimited = True
act.forcerange[:] = [-MAX_MOTOR_FORCE, MAX_MOTOR_FORCE]      # 100
act.ctrllimited = True
act.ctrlrange[:] = [-MAX_MOTOR_VELOCITY, MAX_MOTOR_VELOCITY]  # 5
```

Failure #5: No joint damping or armature

Severity: Medium **Location:** failedsim.py - absent entirely

sucessfulsim.py sets dof_damping[dof] = 0.5 and dof_armature[dof] = 0.01 on every hinge. failedsim.py sets neither. Consequences: zero damping + zero armature + contact impulses from dropping the robot produce jitter or blow-up at first contact; velocity servos (once #2 is fixed) become underdamped and ring; and contact-solver stability degrades because the effective joint inertia is very small.

Fix: Add the for j in range(model.njnt): … if HINGE: … loop from sucessfulsim.py.

Failure #6: No contact-solver tuning for the floor

Severity: Medium **Location:** failedsim.py, step 8

Only collision masks are configured. sucessfulsim.py also tunes contact parameters on the floor geom:

```
model.geom_solref[i] = [0.02, 1.0]
model.geom_solimp[i] = [0.90, 0.95, 0.001, 0.5, 2.0]
model.geom_friction[i] = [1.0, 0.005, 0.0001]
```

Without these, the robot dropped to exactly -min_z (no FLOOR_CLEARANCE) typically sinks a few mm or bounces on the first step. With unstable default solrefs, contact penetration can be severe enough that the robot is launched.

Fix: Add the solref / solimp / friction block; shift the initial z by -min_z + FLOOR_CLEARANCE (0.003).

Failure #7: min_z search only handles mesh geoms, no clearance

Severity: Low **Location:** failedsim.py, step 9

```
if model.geom_type[i] == mujoco.mjtGeom.mjGEOM_MESH:
    ...
    min_z = min(min_z, world_v[:, 2].min())
```

If a link uses a primitive collider (box / cylinder / sphere — common in URDF collision tags), it is silently skipped, min_z is biased upward, and the robot is dropped below the floor at startup. sucessfulsim.py falls back to geom_xpos[2] - geom_rbound for non-mesh geoms, and adds FLOOR_CLEARANCE.

Fix: Add the else fallback and the clearance margin.

Failure #8: Default integrator (Euler) left in place

Severity: Low **Location:** failedsim.py — physics configuration

sucessfulsim.py sets model.opt.integrator = mjINT_IMPLICITFAST. failedsim.py keeps the default Euler. Combined with #5 and #6, Euler at the default timestep is much more prone to energy injection and runaway at the first contact. This bites the moment #1 is fixed and the robot actually falls.

Fix: Set mjINT_IMPLICITFAST, matching sucessfulsim.py.

Failure #9: Duplicate collision geometry

Severity: Low **Location:** failedsim.py, step 3

```
for vis in link.findall("visual"):
    col = ET.SubElement(link, "collision")
    ...
```

sucessfulsim.py guards this with if existing_collisions: continue. failedsim.py unconditionally appends a <collision> per <visual>, even on links that already declare collisions. This produces doubled contacts, duplicated constraint rows, higher effective contact stiffness, and worse solver spikes (compounded by #5/#6).

Fix: Skip links that already have a <collision> child.

What does work in failedsim.py

- URDF package:// path stripping matches sucessfulsim.py.
- Attaching the URDF spec to a new worldbody frame is fine.
- The copy / ET patching logic is fine apart from Failure #9.
- Viewer setup, camera, and main loop are fine.
- On a benign scene/URDF (the free joint happens to land correctly, and the user only wants a static viewer), the script will produce a robot standing on the floor. Hence this is a silent failure, not a crash.

Recommended remediation order

#	Failure	Fix cost	Priority
1	Free joint added after attach()	Move ~6 lines	**Blocker**
2	Actuator is a force motor	Add gaintype, biastype	**Blocker**
3	qpos[2] literal index	Use jnt_qposadr	High
4	No forcelimited / ctrllimited	~4 lines	High
5	No damping / armature	5-line loop	Medium
6	No contact tuning	solref/solimp/friction + clearance	Medium
7	Mesh-only min_z, no clearance	Add else + clearance	Low
8	Default integrator	1 line	Low
9	Duplicate collision	2-line guard	Low

After fixes #1–#4, failedsim.py behaves equivalently to sucessfulsim.py for driving the sliders. Fixes #5–#8 are needed for equivalence on all provided URDFs/scenes; #9 is latent and only manifests on URDFs with explicit collision tags.